Week 9 – Atomic Red Team applied to *Port Scanning and Reconnaissance*

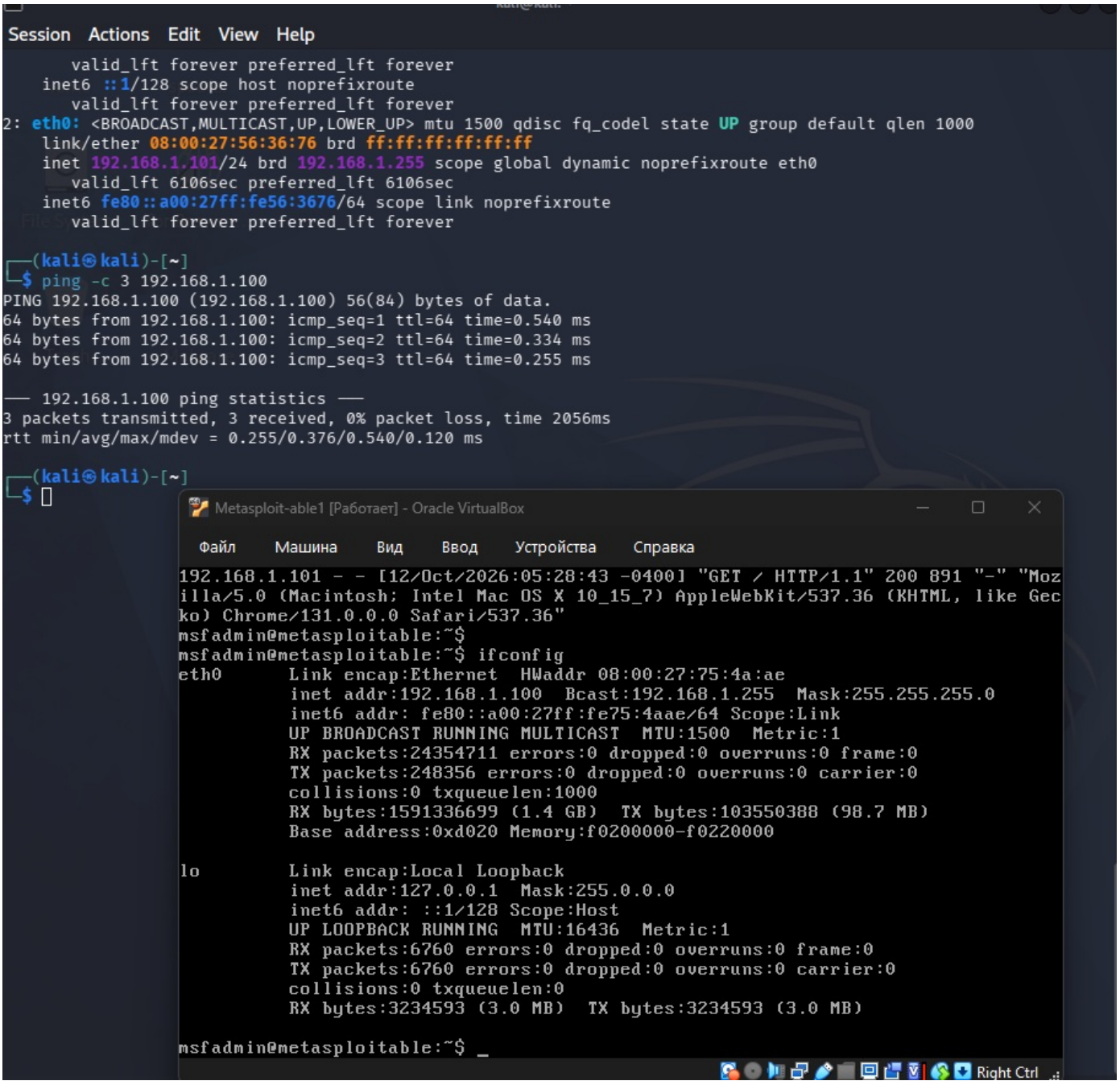
Course: Introduction to Threat Hunting (ITH), 2026-2027, syllabus section 3.3, Week 9 **Syllabus task:** *Run selected atomic tests (e.g., T1059, T1003). Record and analyze the test results in a SIEM.* **Our topic:** Port Scanning and Reconnaissance | **Team:** *<group name / members>*

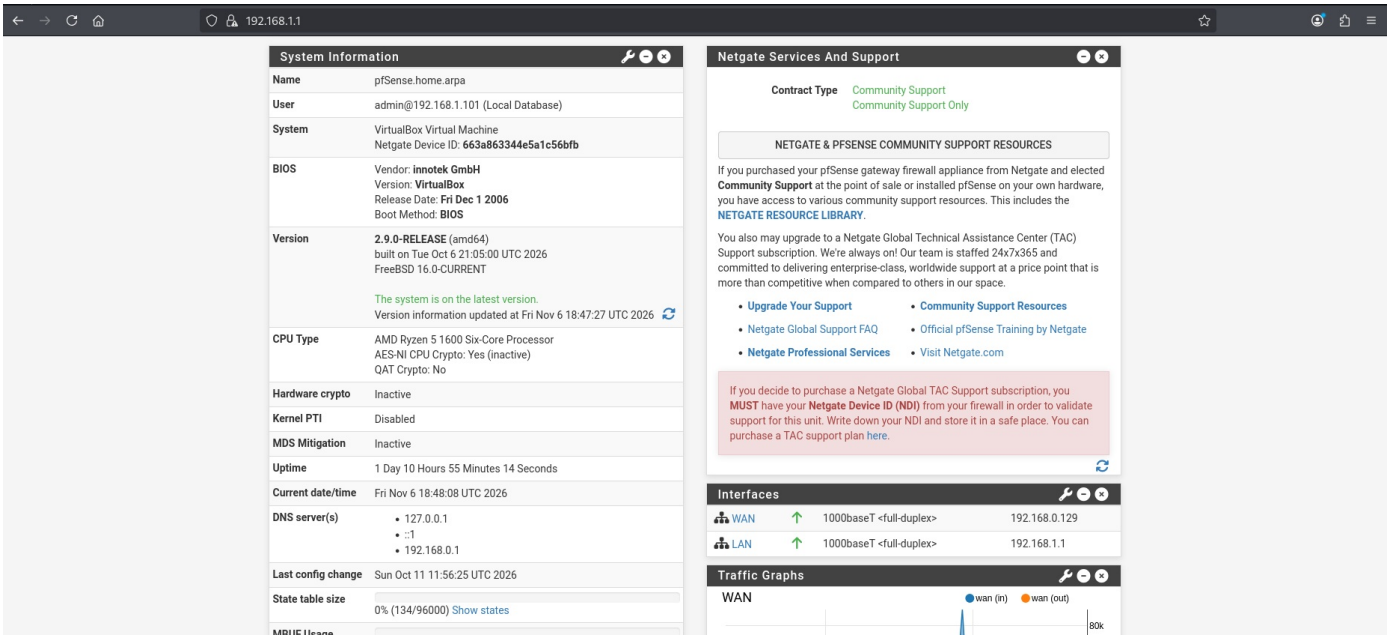
1. Goal and approach

Instead of T1059/T1003 we work with the technique that belongs to our topic: **T1046 – Network Service Discovery** (port scanning once the actor is inside or next to the target network). The same behaviour performed from outside is **T1595.001 – Active Scanning: Scan IP Blocks**. We (1) run TCP SYN scans against a vulnerable host, (2) capture the traffic, (3) load it into a SIEM (Splunk) and (4) analyse it with detection logic.

2. Lab environment (VirtualBox)

Item	Value
Attacker	Kali Linux, **192.168.1.101** (nmap 7.991, Wireshark/tshark, Splunk Enterprise 10.6.1.1)
Target	Metasploitable 2, **192.168.1.100**
Gateway	pfSense 2.9.0-RELEASE, LAN **192.168.1.1**, WAN 192.168.0.129
SIEM	Splunk Enterprise 10.6.1.1 on Kali, data uploaded from the Wireshark CSV export (`scan.csv`)

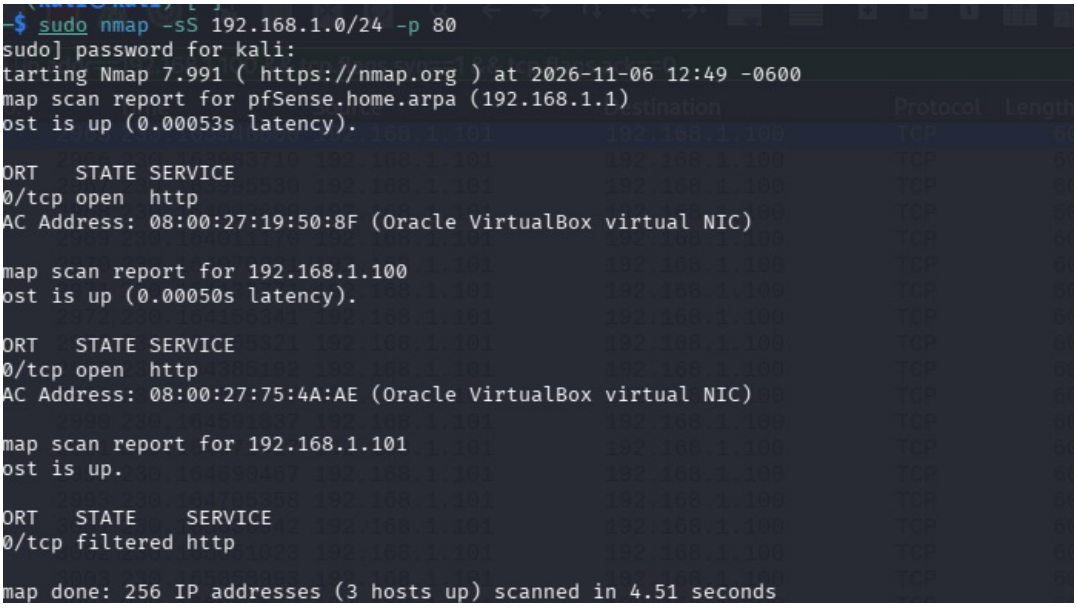




3. Atomic tests for T1046

Atomic Red Team's `atomics/T1046` lists 13 tests (checked on GitHub, raw `T1046.md`). Both Linux tests were executed in the lab, manually:

#	Test	Command	Result in our lab
1	Port Scan (bash `/dev/tcp`)	`for port in {1..65535}; do (2>/dev/null echo >/dev/tcp/192.168.1.100/$port) && echo port $port is open ; done`	**30 open ports** found (full 1-65535 range)
2	Port Scan Nmap	`sudo nmap -sS 192.168.1.0/24 -p 80`	256 addresses, **3 hosts up** in 4.51 s; port 80 open on pfSense (192.168.1.1) and Metasploitable (192.168.1.100), filtered on Kali itself



```
┌──(kali㉿kali)-[~]
└─$ ping -c 3 192.168.1.100
PING 192.168.1.100 (192.168.1.100) 56(84) bytes of data.
64 bytes from 192.168.1.100: icmp_seq=1 ttl=64 time=0.540 ms
64 bytes from 192.168.1.100: icmp_seq=2 ttl=64 time=0.334 ms
64 bytes from 192.168.1.100: icmp_seq=3 ttl=64 time=0.255 ms

── 192.168.1.100 ping statistics ──
3 packets transmitted, 3 received, 0% packet loss, time 2056ms
rtt min/avg/max/mdev = 0.255/0.376/0.540/0.120 ms

┌──(kali㉿kali)-[~]
└─$ bash
┌──(kali㉿kali)-[~]
└─$ for port in {1..65535}; do (2>/dev/null echo >/dev/tcp/192.168.1.100/$port) && echo port $port is open ; d
e
port 21 is open
port 22 is open
port 23 is open
port 25 is open
port 53 is open
port 80 is open
port 111 is open
port 139 is open
port 445 is open
port 512 is open
port 513 is open
port 514 is open
port 1099 is open
port 1524 is open
port 2049 is open
port 2121 is open
port 3306 is open
port 3632 is open
port 5432 is open
port 5900 is open
port 6000 is open
port 6667 is open
port 6697 is open
port 8009 is open
port 8180 is open
port 8787 is open
port 39461 is open
port 40588 is open
port 41613 is open
port 51011 is open
```

Test #1 open ports (from the screenshot): 21, 22, 23, 25, 53, 80, 111, 139, 445, 512, 513, 514, 1099, 1524, 2049, 2121, 3306, 3632, 5432, 5900, 6000, 6667, 6697, 8009, 8180, 8787, 39461, 40588, 41613, 51011.

4. Procedure

1. Check connectivity (`ping -c 3 192.168.1.100`).
2. Run Atomic Test #2 (`sudo nmap -sS 192.168.1.0/24 -p 80`) and Atomic Test #1 (bash loop) from Kali.
3. Start a Wireshark capture on Kali and run two more nmap scans: `sudo nmap -sS 192.168.1.100` (scan 1) and `sudo nmap -sS -T2 -p 1-200 192.168.1.100` (scan 2).
4. In parallel on Metasploitable: `sudo tcpdump -ni eth0 -c 50 'tcp[tcpflags] == tcp-syn'` to see the scan from the victim side.
5. Export SYN packets (`ip.addr==192.168.1.100 && tcp.flags.syn==1 && tcp.flags.ack==0`) with Wireshark *File -> Export Packet Dissections -> As CSV*, upload `scan.csv` to Splunk (*Settings -> Add Data -> Upload*) and run the search in `evidence/real_lab/splunk_query.spl`.

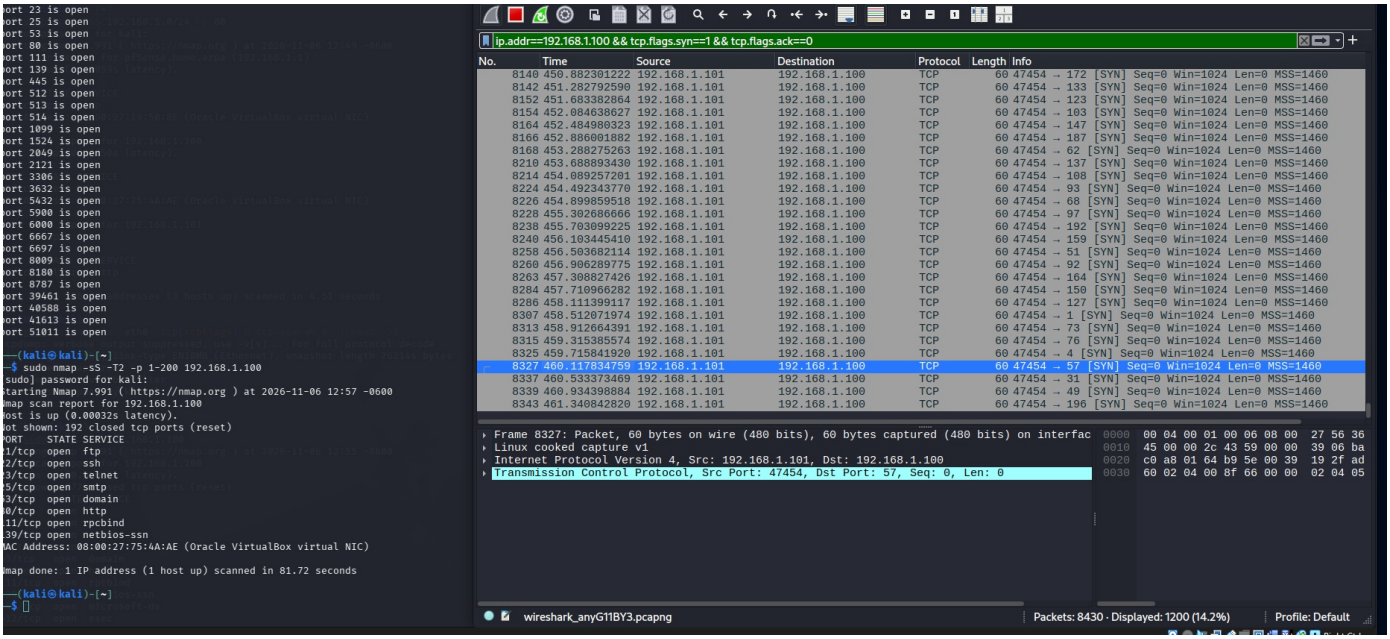
5. Results (real lab)

Scan 1 (`sudo nmap -sS 192.168.1.100`): "Not shown: 977 closed tcp ports (reset)", i.e. 23 open ports among the top 1000. **Scan 2** (`-T2 -p 1-200`): 192 closed ports shown, completed in **81.72 s**.

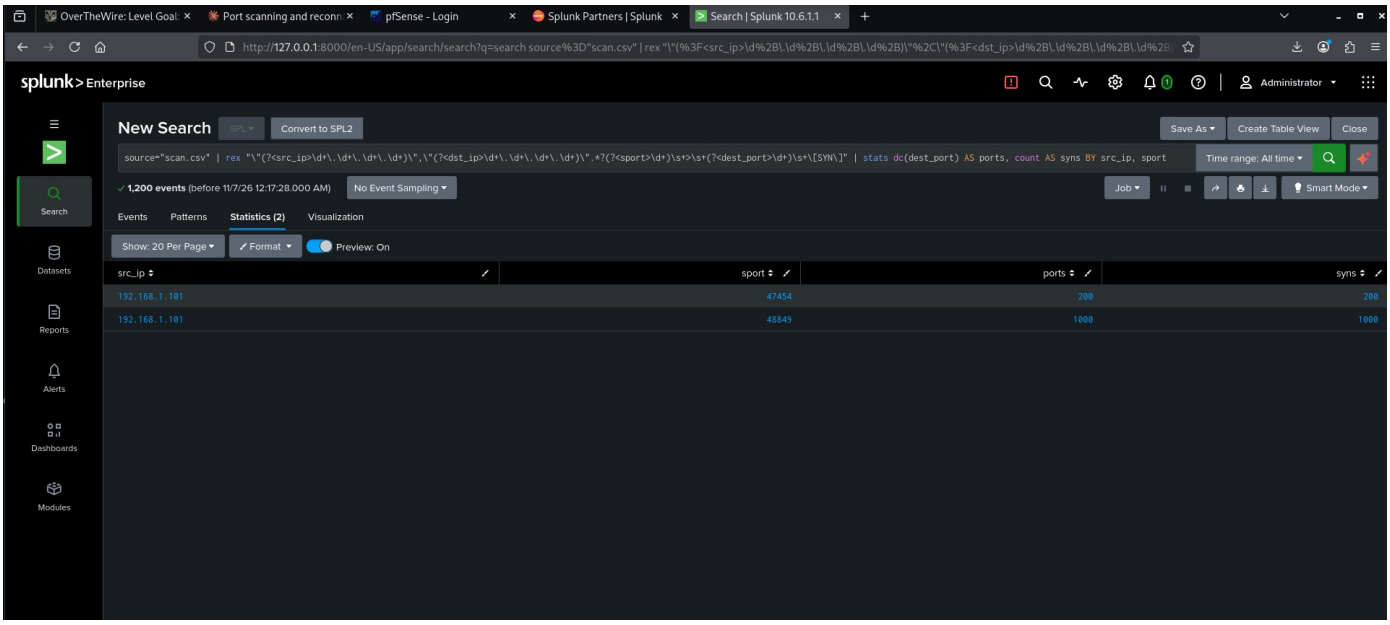
```
┌─$ sudo nmap -sS 192.168.1.100
tarting Nmap 7.991 ( https://nmap.org ) at 2026-11-06 12:55 -0600
map scan report for 192.168.1.100
ost is up (0.00082s latency).
ot shown: 977 closed tcp ports (reset)
ORT     STATE SERVICE
1/tcp   open  ftp
2/tcp   open  ssh
3/tcp   open  telnet
5/tcp   open  smtp
3/tcp   open  domain
0/tcp   open  http
11/tcp  open  rpcbind
39/tcp  open  netbios-ssn
45/tcp  open  microsoft-ds
12/tcp  open  exec
13/tcp  open  login
14/tcp  open  shell
099/tcp open  rmiregistry
524/tcp open  ingreslock
049/tcp open  nfs
121/tcp open  ccproxy-ftp
306/tcp open  mysql
432/tcp open  postgresql
900/tcp open  vnc
000/tcp open  X11
667/tcp open  irc
009/tcp open  ajp13
```

```
Metasploit-able1 [Работает] - Oracle VirtualBox
Файл  Машина  Вид  Ввод  Устройства  Справка
10:00:06.132649 IP 192.168.1.101.48849 > 192.168.1.100.9009: S 2266871267:226687
1267(0) win 1024 <mss 1460>
10:00:06.132655 IP 192.168.1.101.48849 > 192.168.1.100.49165: S 2266871267:22668
71267(0) win 1024 <mss 1460>
10:00:06.132663 IP 192.168.1.101.48849 > 192.168.1.100.8001: S 2266871267:226687
1267(0) win 1024 <mss 1460>
10:00:06.132673 IP 192.168.1.101.48849 > 192.168.1.100.3369: S 2266871267:226687
1267(0) win 1024 <mss 1460>
10:00:06.132682 IP 192.168.1.101.48849 > 192.168.1.100.6: S 2266871267:226687126
7(0) win 1024 <mss 1460>
10:00:06.132688 IP 192.168.1.101.48849 > 192.168.1.100.32781: S 2266871267:22668
71267(0) win 1024 <mss 1460>
10:00:06.132697 IP 192.168.1.101.48849 > 192.168.1.100.9050: S 2266871267:226687
1267(0) win 1024 <mss 1460>
10:00:06.132703 IP 192.168.1.101.48849 > 192.168.1.100.1035: S 2266871267:226687
1267(0) win 1024 <mss 1460>
10:00:06.132739 IP 192.168.1.101.48849 > 192.168.1.100.1687: S 2266871267:226687
1267(0) win 1024 <mss 1460>
10:00:06.132753 IP 192.168.1.101.48849 > 192.168.1.100.407: S 2266871267:2266871
267(0) win 1024 <mss 1460>
50 packets captured
1000 packets received by filter
613 packets dropped by kernel
msfadmin@metasploitable:~$ _
```

On the victim, tcpdump reports **1000 packets received by filter** (50 captured, 613 dropped by the kernel of the old Metasploitable): the whole scan reached the target in a fraction of a second, with source port 48849.



Wireshark: 8 430 packets in the capture, **1 200 displayed** by the SYN filter (14.2 %). Splunk search over `scan.csv`, time range *All time* – 1 200 events, two scans from one source:



Scan	Source	Source port	SYN	Distinct dst ports	Duration	Rate
1 – default nmap SYN scan	192.168.1.101	48849	1 000	1 000	0.073 s	≈ 13 700 SYN/s
2 – `-T2`, ports 1-200	192.168.1.101	47454	200	200	80.3 s	≈ 2.5 SYN/s (one probe per 0.4 s)

Duration and rate come from the first and last timestamp of each scan in the capture (230.1639-230.2368 s and 381.0216-461.3408 s).

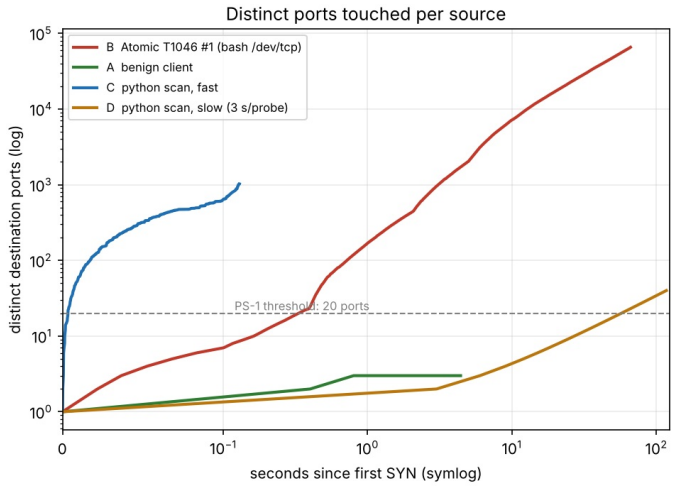
6. Analysis

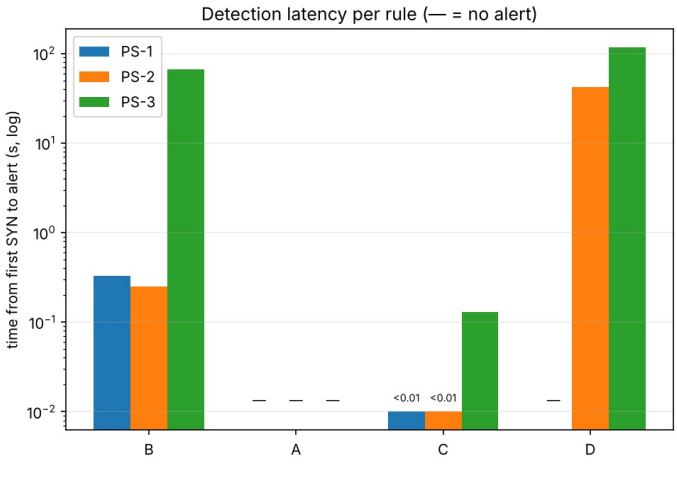
1. **Signature of a raw SYN scan.** Every packet is a bare SYN (length 60, `Win=1024`, `MSS=1460`) from one fixed source port per scan, to a different destination port each time, with no completed handshakes: this is what `nmap -sS` looks like on the wire.
2. **Two scans separated by source port.** Grouping by `src_ip, sport` splits one attacker into two runs, one wide and fast (1000 ports in 73 ms), one narrow and slow (200 ports in 80 s).
3. **Full-range scan finds more than the default scan.** Atomic Test #1 (all 65 535 ports) found **30** open ports; the default nmap scan of the top 1000 ports found 23. The extra ones include high ports (39461, 40588, 41613, 51011) that a top-1000 scan does not cover – services the owner may not know about.
4. **Test #2 is host discovery plus one port.** `-p 80` over the /24 maps which hosts are alive and which run a web server (pfSense and Metasploitable) – discovery of remote systems (T1018) as well as services (T1046).
5. **Detection rules applied to these timestamps** (same logic as `detect/siem/queries.md`):
   - PS-1 (>= 20 distinct ports in 10 s): scan 1 fires after the first 20 probes (< 1 ms); scan 2 fires at ≈ **+7.6 s**, because `-T2` still sends 25 ports in 10 s.
   - PS-2 (>= 15 distinct ports in 120 s): scan 2 fires at ≈ +5.6 s.
   - So `-T2` is **not** slow enough to evade a 10-second rule. A real evasion run needs a much longer delay, e.g. `nmap -sS --scan-delay 3s -p 1-40`, which our automated reproduction (§7) shows is caught only by the 120-second rule.
6. **pfSense cannot see this traffic.** Attacker and target are in the same LAN segment, so packets go through the virtual switch and not through the firewall. Detection therefore relies on the capture on the attacker (Wireshark) fed into Splunk. A production deployment needs the sensor on a mirror port or the traffic routed through the firewall.

7. Supplementary: automated loopback reproduction

To make the experiment repeatable, `lab/run_lab.sh` reproduces it on a single Linux host with loopback addresses: Atomic T1046 Test #1 (the bash loop above, 65 535 ports in 66 s), a fast and a slow Python scanner and a benign client, with a sensor writing JSON events and `detect/detect_scan.py` applying

rules PS-1/2/3.





Run	SYNs	Duration	Alerts
A benign client	12	4.4 s	none
B Atomic T1046 #1	65 535	66.1 s	PS-1 (0.33 s), PS-2 (0.25 s), PS-3
C python fast	1 024	0.13 s	PS-1, PS-2, PS-3
D python slow (3 s/probe)	40	117 s	**PS-2 (42 s)**, PS-3; PS-1 missed

This run is on loopback in an isolated container, not on the VirtualBox lab, and its detection is a Python script, not Splunk.

8. Limitations

- Single attacker and target in one LAN segment; no NAT, no background traffic, no legitimate scanners to allow-list.
- Splunk analysed an exported CSV, not live log forwarding. The Sigma rule and the Splunk/ES|QL queries in `detect/siem/` for PS-1/2/3 were written but only the grouping search in §5 was executed in Splunk.
- Thresholds (20 ports / 10 s, 15 ports / 120 s, 80 % resets) were chosen for this small sample, not tuned on production data.
- The Atomic repository tests were not run through the Invoke-AtomicRedTeam framework; commands were executed manually.
- The Wireshark capture loaded into Splunk (8 430 packets) holds the two nmap scans only. Atomic Tests #1 and #2 are documented by their terminal output (screenshots), not by Splunk events; a full-range Test #1 capture would contain 65 535+ SYN packets.

9. How to reproduce

```
# real lab: capture, export, load into Splunk (see section 4)
sudo tshark -i eth0 -f "host 192.168.1.100" -w /tmp/scan.pcapng
# automated loopback version
sudo ./lab/run_lab.sh && python3 detect/detect_scan.py week9/evidence
```

10. Sources

- Atomic Red Team, T1046 – https://github.com/redcanaryco/atomic-red-team/blob/master/atomics/T1046/T1046.md
- MITRE ATT&CK T1046 – https://attack.mitre.org/techniques/T1046/; T1595.001 – https://attack.mitre.org/techniques/T1595/001/
- Syllabus ITH 2026-2027, section 3.3, week 9.